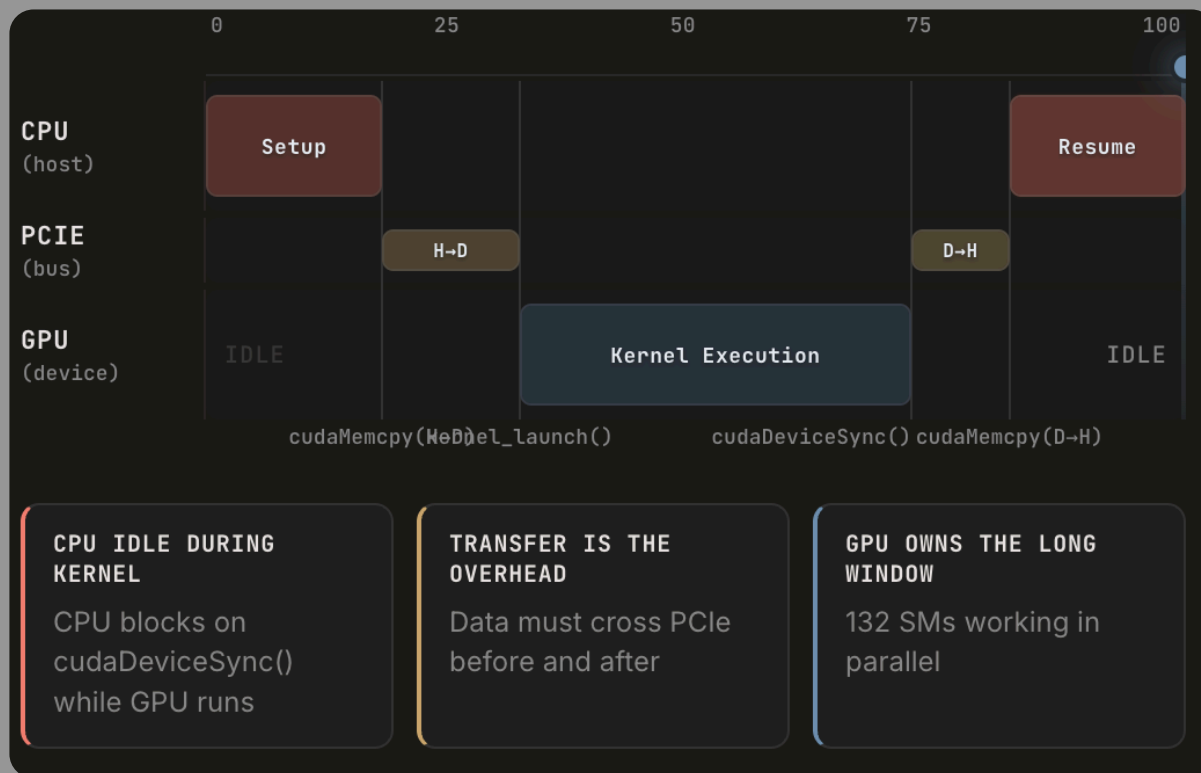


→ The Launch

⇒ Host and Device



> After launch

↳ the GPU runs the long kernel band while CPU thread blocks in `cudaDeviceSync()`

↳ the narrow yellow regions are when bulk data actually rides PCIe in each dir.

⇒ The Kernel

```
vector_add.cu

1 __global__ void add(float* A, float* B, float* C) {
2   int i = threadIdx.x;
3   C[i] = A[i] + B[i];
4 }
```

each thread gets a unique index
→ `threadIdx.x` is the only value that differs between every instance of this kernel

threadIdx.x

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

`i = threadIdx.x`

A[]	3	1	7	2	5	8	4	6	9	2	1	5	3	7	4	8
B[]	2	6	1	9	3	4	7	2	5	8	6	3	9	1	5	2
C[i] = A[i] + B[i]																
C[] (result)	5	7	8	11	8	12	11	8	14	10	7	8	12	8	9	10

showing 16 of 65,536 threads · same code runs on every one

same instruction,
different data
→ every thread executes
line simultaneously
 $C[0] = A[0] + B[0]$
 $C[1] = A[1] + B[1]$
 $\vdots$
 and so on across all
 65,536 threads
 ↳ one instruction
 millions of results
 AT ONCE

⇒ Thread Identity

global index formula:

$$x: \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$

$$y: \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y}$$

blockIdx tells a thread where its block sits in the grid. threadIdx tells it where it sits within that block. Multiply and add so every thread gets a unique global address.

Diagram illustrating thread distribution across a 4x8 grid of blocks, showing the mapping of threads to blocks for different values of bx and by .

The grid is divided into four quadrants, each representing a different combination of bx and by values:

- Top-Left Quadrant ($bx=0, by=0$):** Shows a 4x8 grid of threads. The threads are indexed from 0,0 to 7,3. The threads in the 4th column (index 4) are highlighted in red, indicating they are the active threads for this block.
- Top-Right Quadrant ($bx=1, by=0$):** Shows a 4x8 grid of threads. The threads are indexed from 0,0 to 7,3. The threads in the 4th column (index 4) are highlighted in red, indicating they are the active threads for this block.
- Bottom-Left Quadrant ($bx=0, by=1$):** Shows a 4x8 grid of threads. The threads are indexed from 0,0 to 7,3. The threads in the 4th column (index 4) are highlighted in red, indicating they are the active threads for this block.
- Bottom-Right Quadrant ($bx=1, by=1$):** Shows a 4x8 grid of threads. The threads are indexed from 0,0 to 7,3. The threads in the 4th column (index 4) are highlighted in red, indicating they are the active threads for this block.

The diagram illustrates that for each block, the threads in the 4th column (index 4) are the active threads, corresponding to the bx and by values of the block.

8x4 = 32 threads or one warp per block

$$\text{blockIdx} = (0,0)$$

$$\text{threadIdx} = (4,1)$$

$$\text{blockDim} = (8,4)$$

$$\text{global}(x,y) = (4,1)$$

$$x = 0 \times 8 + 4 = 4$$

$$y = 0 \times 4 + 1 = 1$$

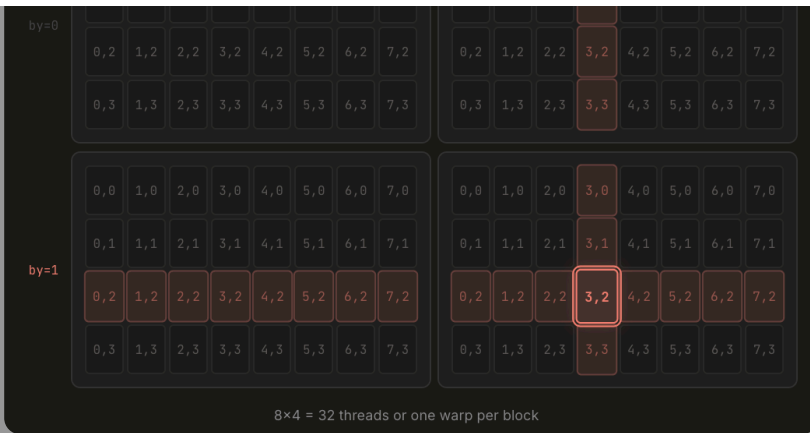
$$\text{flat index: } \text{globaly} * \text{globalWidth} + \text{globalx}$$

$$= 1 \times 16 + 4 = (20)$$

bx=0								bx=1							
0,0	1,0	2,0	3,0	4,0	5,0	6,0	7,0	0,0	1,0	2,0	3,0	4,0	5,0	6,0	7,0
0,1	1,1	2,1	3,1	4,1	5,1	6,1	7,1	0,1	1,1	2,1	3,1	4,1	5,1	6,1	7,1

$$\text{blockIdx} = (1,1)$$

$$\text{threadIdx} = (3,2)$$



$$\text{blockDim} = (8, 4)$$

$$\text{global}(x, y) = (11, 6)$$

$$x = 1 \times 8 + 3 = 11$$

$$y = 1 \times 4 + 2 = 6$$

$$\text{flat index} = 6 \times 16 + 11 = 107$$

→ Hardware Mapping

⇒ The Grid

16 blocks 2048 threads
64 warps 4 warps/block
128 threads/block

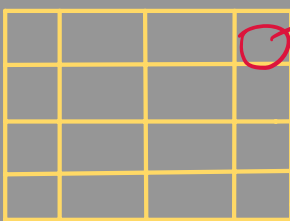
$$\text{Grid} = (4, 4)$$

> each block runs independently on SM
more blocks == more parallelism

> execution happens at the warp level
32-threads vector shares clock & instruction

> all 32-threads run the same instruction simul.
Single Instruction, Multiple Data
(SIMD) exec.

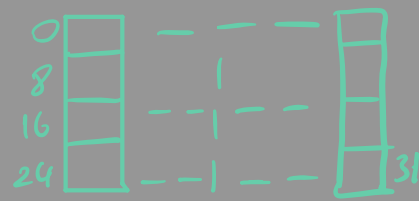
thread
grid



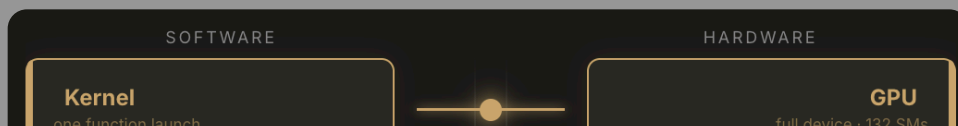
block
structure block(3,0)
4 warps

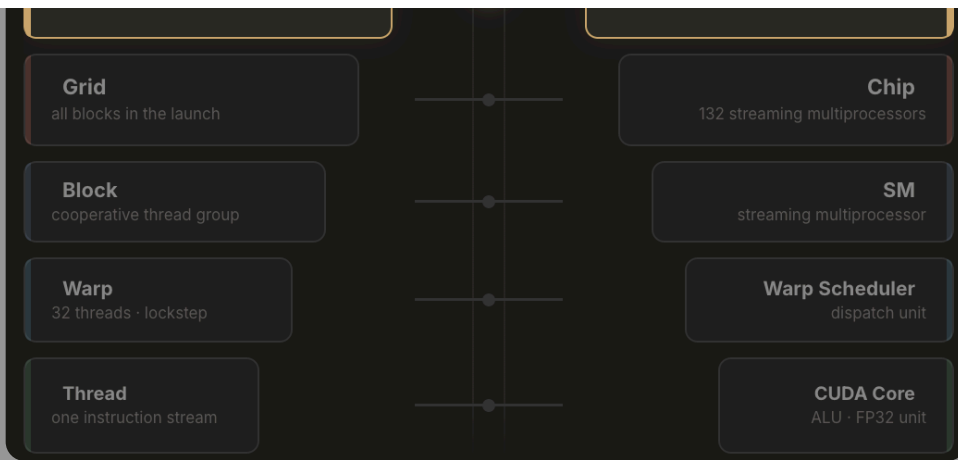
warp 0 --- ... ---
warp 1 --- ... ---
warp 2 --- ... ---
warp 3 --- ... ---
32

warp
execution warp 0
32 threads in parallel

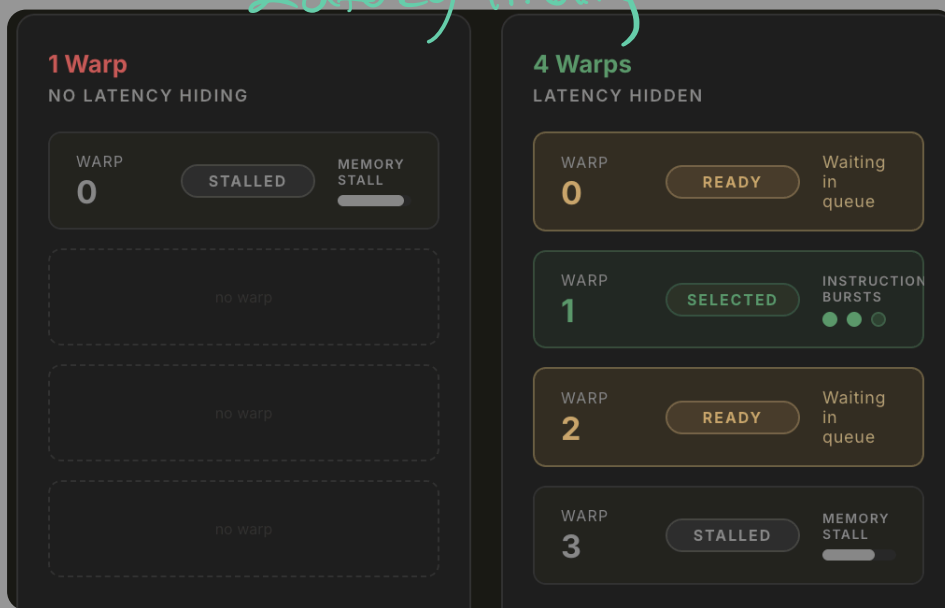


⇒ Software-to-Hardware





=> Warp Scheduler
Latency Hiding



> issues one instruction per clock to a ready warp.
when a warp stalls on memory,
the scheduler skips it and runs another

Compute Unit Utilization

